PrivFuse: Enforcing Privacy SLOs at Admission Time in Multi-Tenant Data Markets

Anonymous Submission
Paper ID: xxxx

***Abstract*—Data trading platforms sell answers computed over personal records contributed by many subjects, and they serve many tenants concurrently. Each subject is promised a bound on cumulative differential-privacy loss, but the promise is enforced today by accounting after the fact: budgets are tallied offline, keyed per tenant, and reconciled in batches. A tenant that splits its traffic across accounts or platforms therefore spends the same subject's budget many times over, and a crash between deciding and recording a query destroys the accounting entirely. We argue that privacy budgets should be enforced at admission, like any other service-level objective, and present PrivFuse, a runtime enforcement layer for multi-tenant data markets. PrivFuse rests on three ideas. First, the hard cap is enforced by an exact durable ledger, while a conservative Count-Min sketch plus a bounded exact table supplies only the windowed pressure signal: estimator memory is independent of the population, and no release decision depends on an approximation. Second, budget accounting is a transaction rather than an increment, so reservation, commit and release are separated behind a subject-striped coordinator that charges each admitted query once even when clients retry, a platform dies mid-query, or the coordinator restarts behind a durable log. Third, windowed SLOs with a hysteretic breaker turn a cumulative bound into predictable degradation. On synthetic populations of up to 10^6 subjects, PrivFuse bounds estimator memory at 10.6 MB where dense exact accounting needs 1.2 GB, admits 0.93 of requests at a cap of ε=1.0 with zero violations where exact per-platform blocking admits 0.98 with 12.7 violations on average, and holds the escape ratio at 0.82 while audit-only policies reach 2.43.**

***Index Terms*—differential privacy, data markets, admission control, privacy accounting, service-level objectives, count-min sketch**

I. Introduction

A data trading platform does not sell records. It sells answers: counts, summaries and trained statistics computed over personal data that many subjects contributed and that many tenants query concurrently. Blockchain-backed designs have made the settlement half of this exchange almost routine, with smart contracts escrowing payment and releasing keys against delivery [1]. The privacy half has not kept pace. A platform can prove that a tenant paid, and that a delivered object matches a committed digest, but it cannot say whether one more answer would push some subject past the privacy budget promised when the data was collected—a question about the accumulated transcript of every query that touched that subject, from every tenant and platform.

Differential privacy (DP) supplies the language for this kind of statement [2], [3], and its guarantee is compositional: charging each release against a per-subject budget bounds the leakage of the whole transcript [4], [5]. Governance treats the budget as a policy parameter rather than a running quantity that must be measured, reserved and settled [6]: it is compiled into documents, tallied in offline jobs and reconciled later, while auditing tools estimate after the fact whether released artifacts respected it [7], [8], [9].

Retrospective accounting is defensible for a single curator releasing a fixed set of statistics; it fails in a market for three reasons that compound. The accounting unit is wrong: budgets are keyed by tenant because the tenant pays, but the loss accrues to the subject, so a tenant that opens accounts at several platforms converts one budget into many. The decision point is wrong: by the time an audit reports an overspend the answers have been released. And the bookkeeping is not a counter but a transaction: a crash between admission and settlement either loses the charge, so the next query is admitted against a consumed budget, or duplicates it, so availability collapses. Our measurements make the first failure concrete: a tenant that splits a budget across 60 accounts achieves an escape ratio of 2.43 against audit-only enforcement and 1.69 against exact per-platform blocking, and neither number is visible to the platform until after the fact.

The obvious repair—keep an exact counter for every subject—does not survive scale. At 10^6 subjects the counter table alone needs 1.2 GB, it must be consulted on the critical path of admission, and it must be updated atomically, which serialises the coordinator. We take a different route, and it rests on one observation: admission control does not need to know a subject's spend, only whether it has crossed a threshold. That predicate tolerates error in one direction and not the other. An over-estimate costs availability, because a safe query is refused; an under-estimate costs soundness, because a cap is breached. If the estimator is one-sided—never below the truth—the platform can trade availability for memory without trading away the guarantee.

PrivFuse turns this into a runtime enforcement layer. A conservative Count-Min sketch over (subject, cost) keys answers threshold queries in constant memory and is one-sided by construction, because its update rule raises only the cells that fall below the running minimum. A bounded exact table holds the heavy hitters, whose budgets are the ones that bind, and a subject returning after eviction is seeded with the sketch's upper bound for its past so that the exact path never starts below the sketch. A windowed SLO evaluator and a hysteretic breaker turn a cumulative bound into a predictable degradation curve, and a two-phase settlement protocol with a subject-striped coordinator makes admission

atomic, idempotent, and robust to a crash between reserve and commit. The composition algebra is zero-concentrated DP, chosen not for tightness but because it is closed-form and invertible: admission must price a query and calibrate a target budget in tens of microseconds at the median, with a millisecond tail, and a Rényi grid capped at $\alpha_{\max}{=}512$ cannot calibrate small targets at all (Section VII-G).

This paper makes four contributions.

1) We formulate privacy enforcement in a data market as an admission-control problem with service-level semantics: a per-subject cap over a sliding window, a tenant model in which the adversary is the splitter, and four objectives (soundness, availability, stability, atomicity) that must hold together (Section III).
2) We give a one-sided sketch accountant whose estimator memory is independent of the subject population, prove that it never underestimates spend, and show that hot-table seeding preserves one-sidedness across eviction (Section IV-C, Proposition 1). The ledger, not the sketch, is the authority for the hard cap; one-sidedness matters because the breaker must not act on a windowed estimate that is too low.
3) We recast budget accounting as a transaction and give a striped coordinator with reserve/commit/release, a transaction identifier bound to a fingerprint of tenant, platform, query digest, support, charge, mechanism and policy version, and a per-identifier lock that makes one identifier mean one payload even across disjoint subject stripes. This establishes single-charge settlement and cap safety under concurrent and crashing clients (Section IV-E, Propositions 2 and 3).
4) We implement PrivFuse as a reference system and evaluate it in twenty experiments, from a twelve-seed cap sweep and a closed-loop attacker to a fault-injection study and a scaling study at 10^6 subjects (Section VII).

The measurements separate what the design buys from what it costs. At a cap of $\varepsilon{=}1.0$, PrivFuse admits 0.934 of requests with zero hard-cap violations, while exact per-platform blocking admits 0.978 and violates the cap 12.7 times per run: a 4.4-point availability gap is the price of soundness. Estimator memory falls from 1.2 GB of dense window state to 10.6 MB at 10^6 subjects, independent of how much of the population is active; a sparse exact table is smaller than the sketch below roughly 2% activity and larger above it, which bounds where the sketch earns its place. We also report where the design does not help: striping pays off only for disjoint subjects, churn erodes the hot table, the monitored soft rate under-reports the population when violators are not heavy hitters, and the periodic monitor sweep dominated the p99 until we replaced it.

II. Background and Motivation

A. Differential Privacy and Its Accounting

A mechanism M satisfies (ε, δ)-differential privacy [10] if for every pair of neighbouring datasets D, D' and every event S, $\Pr[M(D) \in S] \le e^{\varepsilon} \Pr[M(D') \in S] + \delta$. The definition is per-release, but composition carries it to a transcript, which is what lets a platform promise a bound on *total* leakage rather than on each answer, and why that promise needs bookkeeping. Rényi DP [4] gives sharp constants for adaptively chosen mechanisms, at the cost of a discrete order grid. Zero-concentrated DP [5] bounds the Rényi divergence of order $1{+}\alpha$ by $\rho\alpha$ for all $\alpha > 1$, composes by adding ρ, and converts in closed form,

$$\varepsilon = \rho + 2\sqrt{\rho \ln(1/\delta)}, \tag{1}$$

with inverse $\rho = \left(\sqrt{\ln(1/\delta) + \varepsilon} - \sqrt{\ln(1/\delta)}\right)^2$. Equation (1) is not the tightest known conversion—f-DP and Gaussian DP sharpen composition for Gaussian mechanisms [11]—but it is monotone, invertible and cheap, which is what an admission decision needs.

B. The Market Setting

We consider a market in which subjects contribute records to one or more platforms and tenants buy answers over those records. A permissioned ledger records contributions, charges and settlements, so a tenant can verify that an answer corresponds to a committed query—trustless trading provides this for delivery [1] and provenance mechanisms extend it to derived artifacts [12]. What the ledger does not provide is a subject's remaining budget, which depends on queries answered elsewhere and on charges still in flight.

Three properties shape the design. The population is large and heavy-tailed: most subjects appear in a handful of queries, a minority in thousands, and it is that minority whose budgets bind. Tenants are numerous, mutually distrustful and incentivised to maximise information per unit of budget, including under several identities. Queries arrive continuously, so enforcement sits on the critical path of every answer.

C. Approximate Counting for Thresholds

A Count-Min sketch stores a $d \times w$ matrix of counters and updates, for key x and weight v, one cell per row at positions given by d independent hash functions [13]; the estimate is the minimum of the d cells, so collisions can only inflate it. Conservative update raises each cell to at most $m + v$ for the current row minimum m, which never lowers an estimate and reduces over-estimation in practice [14], [13]. A space-saving summary over the same stream retains the k largest keys exactly. Learned variants replace the hash functions [15]; all of them optimise estimation error, whereas our requirement is weaker and, in one respect, stricter: the estimate must never fall below the truth, because the breaker acts on it.

D. Service-Level Objectives

An SLO is a predicate over a measurement window, not over a lifetime total, and a subject's tolerance is typically the sliding version: enforcing only the total exhausts the budget in the first hour, enforcing only the window lets the total drift. PrivFuse maintains both: a cumulative sketch that never forgets, and a ring of epoch sketches for windowed queries, with the SLO on the window and a separate hard cap on the total.

III. Threat Model and Problem Formulation

A. Entities, Trust and Adversary

A market instance comprises subjects $\mathcal{S}$ with $|\mathcal{S}| = N$, platforms $\mathcal{P}$ with $|\mathcal{P}| = n_p$, tenants $\mathcal{T}$, and an append-only ledger $\mathcal{L}$. Each subject s contributes records to one or more platforms and carries a declared cap E_s on cumulative privacy loss, with an SLO saying how the cap may be spent over time. Formally, subjects are identity atoms $u \in \mathcal{U}$, a record is a triple of platform, atom and value, and two datasets are neighbouring if they differ in one record, so a query touching u on two platforms is charged twice against E_u. Identity resolution lies outside the trust boundary: the coordinator accepts a stable pseudonym from the platform's record index and *refuses* a record that arrives without one, so an unknown identity is never a free identity. Each tenant submits queries naming a predicate and the platforms to evaluate it on; the receiving platform prices the query, obtains an admission decision, evaluates the noisy answer, and settles the charge.

The coordinator is *crash-faulty but not Byzantine*: it may stop and lose volatile state, but it does not fabricate charges. The ledger is append-only and authenticated; anchoring it on a public chain strengthens non-repudiation without changing any argument here. Platform code computes the declared mechanism, with attestation left to orthogonal infrastructure [1], [12]. $\mathrm{supp}(q)$ follows from the predicate and the platform's record index, never from private values, so a refusal reveals only that a budget is spent. Out of scope are platform–subject collusion, timing side channels, and a platform that answers with less noise than it charges for, a verifiable-computation problem.

The adversary is a tenant controlling $k \geq 1$ accounts, possibly over several platforms, choosing queries adaptively as a function of the answers received, and aiming to extract more about a target subject than its cap allows. Because the ledger charges accounts rather than subjects, its natural strategy is *splitting*: divide a sequence that together exceeds E_s into k fragments, each within the per-account allowance, and issue them under distinct identities. Splitting is what a rational tenant does when the price of a query is sub-linear in the information it yields. The adversary may also crash the coordinator: charging before execution and crashing loses the charge, charging after execution and crashing delivers the answer for free. Either direction breaks the accounting without any cryptographic failure, which is why settlement is part of the privacy mechanism rather than a billing detail.

B. Cost Model

Every query q carries a cost vector $\mathbf{c}(q) = (\varepsilon_{\mathrm{pure}}, \rho, \delta)$ describing the loss charged to *each* subject in its support. For a Gaussian mechanism with sensitivity Δ and scale σ, $\rho = \Delta^2/(2\sigma^2)$ and $\varepsilon_{\mathrm{pure}} = 0$; for a Laplace mechanism with scale b, $\varepsilon_{\mathrm{pure}} = \Delta/b$ and $\rho = 0$. Charges add in ρ under zCDP composition and in $\varepsilon_{\mathrm{pure}}$ under pure DP composition, so the cost of a transcript is the vector sum of its queries. Write $\sigma_s(t)$ for the cumulative vector charged to s and

$$\sigma_s(t, W) = \sum_{q\,:\, s \in \mathrm{supp}(q),\ t_q \in (t-W,\, t]} \mathbf{c}(q) \tag{2}$$

for the spend inside a window of length W. The hard cap is scalar: a cumulative vector c is within budget when $g(c) \leq E_s$, where $g(c) = \varepsilon_{\mathrm{pure}} + \rho + 2\sqrt{\rho \ln(1/\delta)}$ converts a pure-DP part and a zCDP part to ε at the market's δ. Both mechanisms we price carry $\delta{=}0$ in the vector—Laplace is pure, Gaussian is zCDP—so the conversion consumes δ once rather than once per query. The test must be on g, not on the two components: at a 1.0ε cap, admitting while $\varepsilon_{\mathrm{pure}} \leq 1$ and $\rho \leq 0.021$ separately lets a mixed stream reach a converted 1.41ε, whereas the scalar test rejects the 13 queries that cause it (E24). The SLO constrains the windowed spend under the same scalarisation.

C. The Enforcement Problem

An enforcement mechanism is a function $a : (q, t) \mapsto \{\textsc{admit}, \textsc{throttle}, \textsc{reject}\}$ evaluated before q executes. Four properties are required.

Soundness. Write $\mathbf{c}_{\mathrm{exec}}(q)$ for the cost of the mechanism actually executed: $\mathbf{c}(q)$ in NORMAL mode and $\beta\mathbf{c}(q)$ in THROTTLE mode, where $\beta \in (0, 1]$ is the charge scale of Section IV-D. If $a(q, t) \in \{\textsc{admit}, \textsc{throttle}\}$ then for every $s \in \mathrm{supp}(q)$, $g(\sigma_s(t) + \mathbf{c}_{\mathrm{exec}}(q)) \leq E_s$, and no executed transcript ever produces $g(\sigma_s(t)) > E_s$. The two modes are covered by one condition because a throttled query is still answered: what changes is the noise, and the charge follows the noise. Soundness is what distinguishes enforcement from auditing: it is a statement about the transcript that is never released, not about a sample of past releases.

Availability. Let A be the admitted fraction and U the total value of admitted answers. A mechanism should maximise A and U subject to soundness, and in particular should not refuse a query that is provably safe.

Stability. Let τ be the number of mode transitions per epoch; oscillation is a cost not captured by A, because a tenant that cannot predict admission cannot plan, so τ must be bounded by the hysteresis and dwell parameters.

Atomicity. For every execution, including those with crashes and concurrent submissions, the charge committed to a subject equals the sum of costs of the *committed* transactions that name it, each transaction is charged at most once, and no in-flight reservation is counted as available capacity. A transaction that is admitted but fails before its commit is released and never charged, so an admitted-and-lost answer costs the subject nothing; the converse case, a committed charge whose answer is withheld by the crash, is permitted and counted as over-charge.

The SLO specification is $(\theta_{\mathrm{soft}}, p, W)$ with $\theta_{\mathrm{soft}} = \alpha E_s$ for a soft ratio $\alpha \in (0, 1)$: the fraction of subject-epochs whose windowed spend exceeds θ_{soft} must not exceed p, and no subject-epoch may exceed E_s. The two thresholds are enforced by different mechanisms and are deliberately separate. The

hard cap is enforced by the exact ledger at reservation, so it does not depend on the estimator. The soft threshold is a target the breaker pursues using the estimator's windowed pressure; the monitor samples the population (Section VII-B), so p is *best effort* in the population and exact only for the monitored subjects, where the ledger still guarantees the cap.

D. Success Criterion for an Attacker

To compare mechanisms under splitting we report the *escape ratio*

$$R = \max_{s \in \mathcal{S}} \; g\big(\sigma_s(T)\big)/E_s, \tag{3}$$

where T is the trace length and $\sigma_s(T)$ the total charged to s over it: the worst-case ratio of loss to declared cap. $R \leq 1$ certifies soundness on the trace; $R > 1$ quantifies the leak. The ratio folds the effect of splitting, of per-account keying and of the composition rule into one quantity comparable across mechanisms.

IV. Design

A. Overview

PrivFuse sits between the query planner and the execution engine of every platform, and it holds two kinds of state that answer different questions. The *settlement ledger* answers "what has this subject been charged, and what is still in flight?". It is exact, it is the authority for the hard cap, and it has no time structure. The *estimator* answers "how fast is this subject's budget being consumed, relative to the window?". It has time structure, it is consulted on every request, and it may be wrong in one direction only.

The split is deliberate. Exactness is cheap where the state is one scalar per subject and expensive where it is one counter per subject per epoch: a thirty-day window at five-minute resolution needs 8,640 epochs, so an exact windowed accountant that materialises every (subject, epoch) pair needs 8.6×10^9 counters; a sparse one keeps only the epochs a subject appears in, as E16 does, and then pays per event rather than per subject. We therefore make the ledger exact and the estimator one-sided, and Section V proves that the composition preserves both properties.

Figure 1 shows the request path. A query arrives with a declared sensitivity and target ε and the pricer converts it into a cost vector; the breaker combines subject pressure from the estimator with tenant pressure from the observed soft-violation rate and selects a mode. In NORMAL mode the full cost is reserved, in THROTTLE mode the query is answered at a reduced charge (more noise, less utility), and in REJECT mode it is refused without touching data. Reservation is two-phase, and only a committed charge reaches the estimator, so a failed execution costs the subject nothing. The ledger, not the estimator, is the authority for the hard cap; the estimator supplies the windowed pressure the breaker acts on, at memory independent of the population.

B. Pricing a Query

A query plan declares a sensitivity upper bound Δ and a target per-query ε_q at a fixed δ. Under zCDP the platform answers with a Gaussian mechanism whose scale is $\sigma = \Delta/\sqrt{2\rho_q}$, where ρ_q inverts (1) at ε_q; a Laplace mechanism is priced with $\varepsilon_{\mathrm{pure}} = \Delta/b$ and $\rho = 0$. The resulting $\mathbf{c}(q)$ is the unit of account for every structure that follows, and the inversion is what lets a tenant ask for "ε_q=0.25 per query" and receive a price in ρ that composes without further conversion. Pricing is conservative: the pricer takes an upper bound on sensitivity, because an under-priced query is an unbounded liability while an over-priced one merely triggers the breaker early. The controller never re-prices between reservation and commit, so a crash cannot change the price of an answer already computed.

C. A One-Sided Estimator

The estimator keeps three structures. A *cumulative layer* is a pair of Count-Min sketches, one over $\varepsilon_{\mathrm{pure}}$ and one over ρ, so that mixed pure-DP and zCDP charges compose without conversion on the hot path. An *epoch ring* is a bounded deque of epochs, each with its own pair of sketches, supplying the windowed view of (2). A *hot table* holds the H subjects with the largest accumulated charge exactly since their admission, because those are the subjects whose budgets bind.

For key s and cost c, the conservative update computes $m_j = \min_r A_j[r, h_{j,r}(s)]$ per layer and raises every cell on s's row that falls below $m_j + c_j$ to $m_j + c_j$; the estimate is the minimum over rows. The rule never decreases a cell and never decreases an estimate; raising a cell can raise a colliding key's estimate, which costs availability and never soundness, and it stays closer to the truth than plain increments at the same width [14], [13]. With $d = \lceil \ln(1/\delta_s) \rceil$ rows and $w = \lceil e/\varepsilon_s \rceil$ columns the estimate exceeds the true charge by at most ε_s times the layer's total mass with probability $1 - \delta_s$, and the bound is one-sided: it bounds the excess rather than bracketing an unbiased estimate.

The hot table interacts with the sketch in a way that needs care. An evicted subject that returns must not restart from zero, because the sketch already holds its past. We *seed* it with the sketch's current upper bound and then accumulate exactly. The estimate is therefore the sketch's bound on the past plus the exact charge since re-admission, which still bounds the true total, with excess at most the sketch's error at the moment of eviction. Seeding from below, for instance with the point estimate minus one standard deviation, would restore accuracy at the cost of soundness, and we do not do it.

The ring is sized so that a window is always covered: a window straddling a boundary spans at most $\lceil W/\ell \rceil + 1$ epochs, so R is exactly that. An earlier version used $\lceil W/\ell \rceil$, which dropped the oldest epoch of every straddling window and under-counted by up to one epoch of traffic—a one-line error that turns a one-sided estimator into an unsound one, and one that a purely cumulative test suite does not catch.

Total state is $O(dw)$ for the cumulative layer, $O(Rdw)$ for the ring and $O(H)$ for the hot table, none of which depends

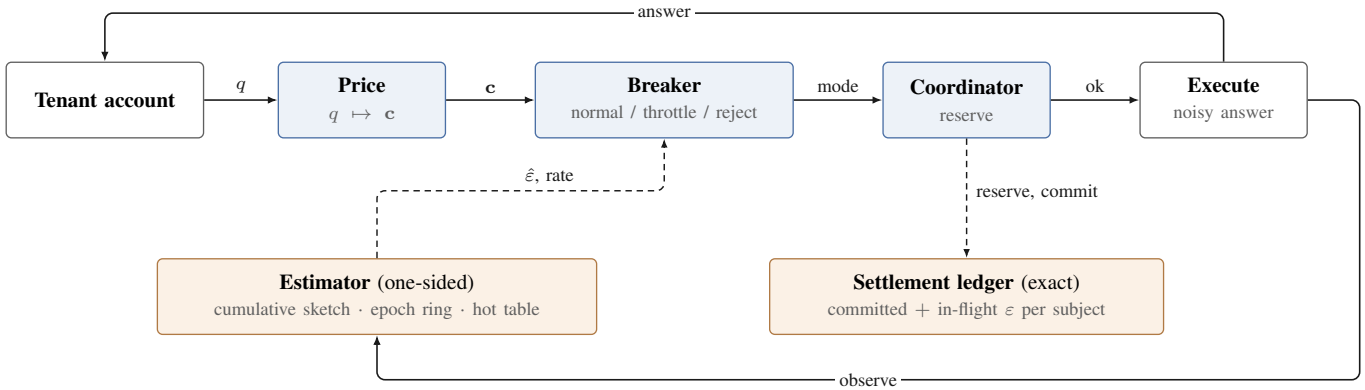


Fig. 1. Request path. Solid arrows carry requests and answers, dashed arrows carry accounting state. The ledger enforces the cumulative cap exactly; the estimator supplies windowed and tenant-level pressure and may over-estimate but never under-estimate.

on N. The default uses $w = 2^{14}$, $d = 4$, $R = 3$ and $H = 1024$, occupying 10.6 MB at 10^6 subjects against 1.2 GB for a dense table, and not growing when the population doubles. The price is an over-estimate that is invisible for heavy hitters—the hot table holds them exactly—and bounded by ε_s times the layer mass for the tail, where the cap is least likely to bind. Section VII-B measures both effects.

D. From a Cap to an SLO

A cumulative cap alone produces the wrong behaviour at both ends: a tenant that spends a subject's budget in the first hour gets full service for an hour and nothing thereafter, while one that spreads the same spend over thirty days gets uniform service. Neither is what the subject agreed to. PrivFuse therefore evaluates a population-level SLO: over a window of length W, the fraction of monitored subjects whose spend exceeds $\theta_{\text{soft}} = \alpha E$ must not exceed p, and no subject may exceed E. The default is $\alpha = 0.8$, $p = 0.01$, $W = 600\,\text{s}$ and $\ell = 300\,\text{s}$ epochs, so the ring holds three epochs.

The monitor reads the windowed estimates of the hot table rather than of the whole population, on the argument that a subject outside the hot set has, by construction, accumulated less than the eviction threshold, so the subjects most likely to violate are the ones the table holds. This keeps the monitor's cost independent of N and concentrates attention where violations are most likely, but the rate is over the monitored set, not the population: a subject outside it can still violate when the H ahead of it do. The loop is best-effort feedback; the hard cap is the guarantee. The tenant-level rate is refreshed every 64 evaluations, bounding staleness at the cost of a lag between a change in behaviour and the breaker's response.

The breaker is a four-level machine: pressure in $[0.60, 0.85)$, $[0.85, 1)$ and $[1, \infty)$ maps to throttle, reject and freeze; escalation is immediate; de-escalation drops one level after d=3 consecutive evaluations below the current band's threshold minus h=0.05. The margin prevents oscillation around a threshold, the dwell prevents one quiet evaluation from releasing a subject about to be charged again. In throttle mode the charge is scaled by $\beta = 0.5$, which inflates Gaussian noise by $1/\sqrt{\beta}$ and Laplace noise by $1/\beta$; the tenant receives a lower-quality answer at a lower price rather than a refusal. This turns a hard wall into a degradation curve, and it is sound because the mechanism executed is the one charged for.

E. Two-Phase Settlement

Reservation is all-or-nothing over the query's subject set, and a rejection is not a partial charge: either the query is reserved against every subject it touches, or it touches no budget at all (Algorithm 1).

Commit is idempotent in the transaction identifier. A repeated commit for an already-committed transaction increments a duplicate counter and returns the existing record rather than charging again, which makes client retries safe. Release returns the reserved budget and is a no-op for a transaction already committed or released. Reservations neither committed nor released within their TTL are expired by a periodic tick, bounding the budget a crashed client can hold hostage to one TTL; the cost is a window in which budget is unavailable although no answer was delivered, so the TTL trades availability under crash against availability under load.

Concurrency is handled by striping. Subjects map to $L = 64$ stripes by a hash of the identifier, and a transaction acquires its stripes in sorted index order, ruling out deadlock; transactions over disjoint subjects then proceed in parallel. A transaction spanning many subjects serialises against everything sharing its stripes, which is correct for a multi-subject query and is why striped and global locking behave alike when all traffic hits one subject. The difference appears when subjects are disjoint, and Section VII-I measures both.

V. Analysis

We state the properties the design guarantees and sketch why they hold. The proofs are short because the mechanisms are chosen to make them short: one-sidedness is an invariant of an update rule, safety of a check-and-record, atomicity of an identifier.

Proposition 1 (One-sidedness)**.** *During normal execution, and after recovery completes, the cumulative estimate satisfies* $\hat{\sigma}_s(t) \succeq \sigma_s(t)$ *for every subject* s *and time* t*, and the*

Algorithm 1 Settlement. The log append is the commit point: durable before the state changes and before any answer leaves. The identifier index and the stripes are taken before the state is read, so one identifier means one payload even when two supports are disjoint. States: RESERVED→COMMITTED (deliver the stored answer), RESERVED→RELEASED (abort or TTL); both are terminal, a repeated commit is a no-op, and a retry with the same identifier but a different payload is refused. $H(\cdot)$ covers tenant, platform, query digest, support, charge, mechanism and policy version.

procedure RESERVE(tx, S, c)
  LOCKTX(tx); lock stripes of S in sorted order ▷ tx first: no cycle
  $st \leftarrow$ STATE(tx)
  if $st \neq$ NONE **then** ▷ retry: the payload must be identical
    if HASH(tenant, platform, q, S, c, mech, v) $\neq$ FINGERPRINT(tx) **then**
      unlock; **return** REFUSE
    end if
    if $st =$ COMMITTED **then**
      unlock; **return** DELIVER
    end if
    if $st =$ RESERVED **then**
      unlock; **return** EXECUTE
    end if
    unlock; **return** REFUSE ▷ released and expired are terminal
  end if
  if $\exists s \in S :\ g(\text{spend}(s) + \text{inflight}(s) + c_s) > E_s$ **then**
    unlock; **return** REFUSE
  end if
  APPEND(RESERVED, tx, S, c); FSYNC
  inflight(s) += c_s for $s \in S$; SETSTATE(tx, RESERVED); unlock
end procedure
procedure COMMIT(tx, r)
  LOCKTX(tx);
  if STATE(tx) $\neq$ RESERVED **then** unlock; **return**
  end if
  APPEND(COMMITTED, tx, r); FSYNC ▷ r: digest of the stored answer
  spend(s) += c_s; inflight(s) −= c_s; OBSERVE(s, c_s)
  unlock; **return** DELIVER
end procedure

windowed estimate satisfies $\hat{\sigma}_s(t, W) \succeq \sigma_s(t, W)$, where the comparison is componentwise on $(\varepsilon_{\text{pure}}, \rho)$. Recovery replays the committed log before the coordinator admits anything; during replay the estimator is not queried, so there is no interval in which a client is served against a partially rebuilt state.

Proof: Maintain, for every layer j and key x, the invariant $\min_r A_j[r, h_{j,r}(x)] \geq T_j(x)$ for the true charge T_j; it holds with all cells zero. An update of x by c sets each cell on x's row to $\max(a, m + c)$ with $m \geq T_j(x)$ the row minimum, so every such cell becomes at least $T_j(x) + c$, and cells of other keys only increase. A hot subject is seeded with the sketch's bound at re-admission, hence at least $T_j(x)$, and then incremented exactly. The windowed estimate sums whole epochs covering the window, each bounded below by its true charge, and charges are non-negative, so the sum bounds the true windowed charge provided the ring holds $\lceil W/\ell \rceil + 1$ epochs, which Section IV-C enforces. ∎

Over-estimation costs availability; Section VII-B measures 0.02% relative error for the top percentile and no false negatives in 24 configurations.

Proposition 2 (Cap safety)**.** *Assume that every platform reserves through the shared coordinator before a query executes, that the coordinator is the only writer of a subject's budget, and that the cost it charges is the cost of the mechanism the platform executes. Then, for every subject s and time t, $g(C_s(t) + F_s(t)) \leq E_s$, the committed charge $C_s(t)$ is non-decreasing, $F_s(t) \succeq 0$, and no executed query leaves $g(C_s(t)) > E_s$—even when tenants split their traffic across accounts and platforms.*

Proof: Let $C_s(t)$ be the committed charge and $F_s(t)$ the sum of live reservations. Admission of q at time t requires $g(C_s(t^-) + F_s(t^-) + \mathbf{c}_{\text{exec}}(q)) \leq E_s$ for every subject in the support, and the test and the insertion of the reservation happen under one stripe guard per subject, so no other transaction observes $C_s + F$ between them. A live reservation of q then undergoes exactly one of three transitions. Commit moves $\mathbf{c}_{\text{exec}}(q)$ from F_s to C_s, which leaves $C_s + F_s$ unchanged; release removes the same vector from F_s; expiry removes it as well. Hence $g(C_s(t)+F_s(t)) \leq E_s$ holds in every reachable state, C_s never decreases and $F_s \succeq 0$, because a reservation leaves F_s the moment it is settled or expires. A query that executes carries a committed charge by Proposition 3, and is charged once, so the loss it incurs is counted in C_s exactly once and cannot exceed E_s. Splitting does not enter the argument because the key of every reservation is the subject, not the account or the platform. ∎

Proposition 3 (Single-charge settlement)**.** *Assuming the ledger outlives the coordinator process, each transaction is charged at most once under crashes and retries, and no answer is delivered without a committed charge. A committed charge whose answer is lost to a crash is permitted, so this is at-most-once delivery with exactly-once charging, not exactly-once delivery.*

Proof: Commit is keyed by the transaction identifier and returns the existing record if the transaction is already committed, so a retry cannot charge twice. The platform calls commit before returning the answer to the tenant, so an answer that leaves the platform has a committed charge; a crash before commit means the answer is never delivered. A reservation that is neither committed nor released is expired after its TTL, which bounds the budget held by a dead client. The protocol can therefore over-charge a subject whose answer was computed but lost to a crash, which is the safe direction, but cannot release data without charging for it. ∎

Proposition 4 (Bounded instability)**.** *For a single breaker key, the number of de-escalations in T evaluations is at most $\lfloor T/d \rfloor$, where d is the dwell parameter.*

Proof: A de-escalation requires d consecutive evaluations below the threshold minus the hysteresis margin, and any other evaluation resets the counter. Each de-escalation therefore consumes at least d distinct evaluations. ∎

The bound is loose—E7 observes 12 de-escalations over

roughly 5,000 evaluations with $d = 3$—but it has the right shape: transitions are controlled by a parameter rather than by the load, and removing hysteresis raises the count to 26.

Two properties are deliberately absent. We do not prove that estimator error stays below a constant as the population grows, because the Count-Min guarantee is relative to total mass; we measure it instead. Nor do we prove that the SLO is met under every arrival process: the breaker is a feedback controller, and an abrupt load increase can drive pressure past the reject threshold before it reacts. The hard cap survives such a transient because an exact structure enforces it.

VI. Implementation

PrivFuse is a reference implementation in Python 3.11: about 6,400 lines across 55 modules with 56 tests. The enforcement core depends only on the standard library; NumPy, pandas and matplotlib are used by the experiment drivers, so the measured per-request cost is not an artefact of a numerical stack.

Two components are incomplete. The ledger has in-memory and write-ahead-log backends, the latter replaying committed charges across a process death (E21), and a Hyperledger Fabric adapter that raises `NotImplementedError`; the safety argument does not depend on the backend, which sees only reserve, commit and release, but we do not measure Fabric latency, so our numbers bound the protocol rather than the settlement path. The attester is null: verifying that a platform ran the mechanism it charged for needs remote attestation or verifiable computation. On restart the 391 committed charges of E21 are replayed into the estimator before the coordinator admits anything, every estimate at or above committed spend, so the windowed view returns with the log rather than empty. Every run is seeded and writes raw rows, so one command regenerates every table and figure; an anonymized artifact accompanies the submission.

VII. Evaluation

A. Methodology

We run on a two-vCPU container with 4 GiB of RAM and CPython 3.11; the host is deliberately small, so absolute latencies are a conservative upper bound. Experiments use three seeds and report the mean, except the cap sweep, which is rerun on twelve; per-seed rows ship with the code.

The generator produces 12,000 requests from 40 tenants over 1,500 subjects and three platforms, with popularity from a truncated Zipf law of exponent 1.15 and 15% of requests touching several subjects; every subject carries a cap of ε=1.0 at $\delta = 10^{-5}$. Identities are drawn by inverse transform, so large ranks fold back onto small ones and the heavy hitters that make accounting hard survive. The splitting attack gives an adversary 12 target subjects, a budget of 0.30ε per target and k fragments from 12 to 60, issued under distinct identities across platforms.

We compare against five policies: *static quota*, a fixed per-platform allowance that meters spend but can neither throttle nor window it; *token bucket*, a per-subject allowance that refills at 0.05ε per second; *audit only*, which admits everything and flags subjects afterwards and is the deployed status quo; *exact blocking*, per-platform counters checked against composed spend; and *exact global*, exact per-subject counters shared across tenants and platforms, with no window, breaker or degradation. PrivFuse is the only policy that is subject-keyed across platforms, time-structured and memory-bounded in its estimator.

B. Is the Estimator Sound and Accurate?

A false negative in the estimate cannot release an over-cap answer, because the ledger refuses it; what an under-estimate costs is a late throttle, which is why the estimator is built one-sided. Across 24 configurations—four widths, with and without the hot table, three seeds—the estimate never falls below the true charge, as Proposition 1 requires, including across eviction and re-admission.

The monitor, not the estimator, limits what the SLO means (E25). At 24 checkpoints of a 600 s trace at cap 0.5 we recompute the exact windowed composition of every subject and compare it with the rate the hot table reports. In the heavy-tailed population the table's recall is 1.0 for $H \geq 64$ and 0.95 at H=8, and the monitored rate matches the population rate to three decimals. Flatten the popularity (Zipf 0.6) and shrink the table to eight slots and recall collapses to 0.014: 70 of 71 violations are missed, and the monitored rate reads 0.000 against a population rate of 0.002, because violators are no longer heavy hitters. Scanning the sketch over all N subjects recovers every violation in both regimes with at most two false alarms in 1,500 subjects—the loss is sampling, not conservatism—but the scan costs $O(N)$ per epoch. The windowed SLO is therefore best effort over the population; the cap is not.

Accuracy separates along the population. For the top percentile by spend—the subjects whose budgets bind—relative error is 4.3% without a hot table at width 512 and 0.02% from width 2048 up, where the table rather than the width does the work. For the tail, mean absolute error falls from 0.071ε to 0.013 and p99 from 0.37 to 0.10; width 32,768 measures zero error, which is why we do not deploy it.

C. What Does It Cost?

The estimator costs 27–32 μs per request across widths from 2^{10} to 2^{16}, against 4.6 μs for an exact table: an 18% rise for a sixteen-fold increase in width, as expected from eight memory accesses per update plus the hot-table lookup. The full admission path is slower and far more variable (Fig. 3a). Its tail came from one function: a tenant-pressure sweep that scanned the hot table every 64 admissions, which cost p99 5.8 ms against 0.9 ms with the sweep removed. We replaced it with an incremental update at commit time—$O(\text{ring})$ per charge, and a monitor over every charged subject rather than the hot table—and the p99 falls to 1.1 ms for the same trace and host, with the mean moving from 0.28 to 0.23 ms (E17).

Queueing still decides the deployment (Fig. 3b): at 50% utilisation the median is 0.26 ms but the p99 is 160 ms; a load

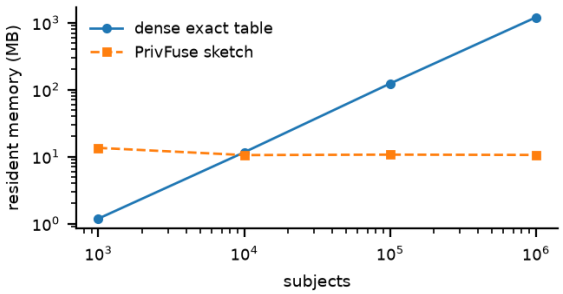


Fig. 2. Resident memory as the subject population grows (E23), with the default four live tables. The dense table grows linearly with N; the sketch is flat. Zero under-estimates at every population.

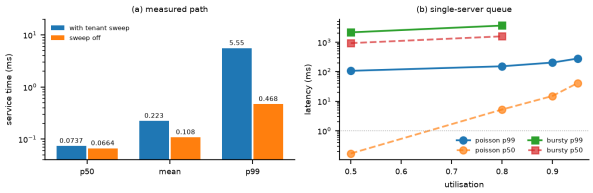


Fig. 3. Admission latency (E17, 12,000 requests, median of three trials). (a) Service time under the per-request pressure sweep and under the incremental monitor we adopt, log scale. (b) FCFS queue over the measured service times under Poisson arrivals and under a burst at five times the average rate.

that averages 50% while peaking at five times capacity makes the median 0.9 s. The path must be provisioned for its peak, and microsecond averages are the wrong number to quote.

Memory tells the opposite story at small scale: below roughly 10^4 subjects a dense table is no larger than the sketch, because the sketch's width is paid once and the table's cost has not caught up (Fig. 2). From 10^3 to 10^6 subjects the dense table grows from 1.2 MB to 1.2 GB while the sketch stays at 10.6 MB (E23); per-request cost rises from 25 to 45 μs for the sketch and 9 to 13 μs for the table, and the estimate keeps its one-sidedness, with maximum absolute error 0.057ε (E8).

The structure answering windowed queries is therefore $O(1)$ in N, whereas a naive windowed accountant—one cumulative structure per epoch—multiplies the measured 1.2 GB by the ring depth. The whole system is not: the ledger keeps one scalar balance per subject that has ever transacted, far smaller than a cost vector, and a deployment would shard it.

D. Does Enforcement Hold?

Table I reports the central comparison at six cap levels, each averaged over twelve seeds. At a cap of ε=1.0, PrivFuse admits 0.934 of requests and produces no hard-cap violation, while exact per-platform blocking admits 0.978 and lets 12.7 subjects exceed the cap on average, the worst of them reaching 1.69ε. Static quota behaves similarly at 0.956 availability and the same violation count. Their counters are exact; the violations come from the key: each platform meters its own share, so a subject with records everywhere is charged once per platform and accumulates up to three times the cap, and natural traffic alone pushes a dozen subjects past it.

TABLE I
ENFORCEMENT ACROSS CAP LEVELS (E15, MEAN OF TWELVE SEEDS). "VIOL." IS THE NUMBER OF SUBJECTS WHOSE CUMULATIVE SPEND EXCEEDED THE CAP; AVAILABILITY IS THE ADMITTED FRACTION. THE 95% CONFIDENCE INTERVAL OF PRIVFUSE'S AVAILABILITY IS AT MOST $\pm$0.008 AT EVERY CAP.

	PrivFuse		Exact blocking		Static quota	
Cap	Avail.	Viol.	Avail.	Viol.	Avail.	Viol.
0.50	0.752	0	0.888	15.2	0.903	15.1
0.75	0.854	0	0.956	13.0	0.939	13.0
1.00	0.934	0	0.978	12.7	0.956	12.7
1.50	0.966	0	1.000	12.0	1.000	12.0
2.00	1.000	0	1.000	12.0	1.000	12.0
3.00	1.000	0	1.000	0.0	1.000	0.0

An adversary that sees the decision can re-plan (E18): halving a fragment after each refusal raises PrivFuse's composed loss from 0.80 to 0.85 of the cap, still inside the hard cap, but raises static quota from 1.30 to 1.74 and exact blocking from 1.69 to 1.76. A smaller request is cheaper and slips under a quota that refused the full fragment; a keyed account still charges it. Spreading attempts over forty tenants changes nothing—identity is not the key—but spreading them over three platforms pays, because each meters its own counter.

The availability gap is the price of soundness and it shrinks as the cap loosens: 13.6 points at 0.5ε, 4.4 at 1.0ε, 3.4 at 1.5ε and nothing at 2.0ε, where the soft threshold stops binding and PrivFuse behaves like exact global accounting while the baselines still violate the cap twelve times per run. The 3.0ε cap is a control: no policy violates it, since the trace's maximum is 2.01ε.

At 1.0ε PrivFuse admits 4.4 ± 0.2 points fewer requests than exact blocking and 2.2 ± 0.2 fewer than static quota, while both exceed the cap 12.7 ± 0.3 times per run against zero; the paired difference is tighter than the interval because every policy sees the same traces, and deterministic accounting means only the violation count varies across seeds. Zero violations is an observation over twelve traces, not a proof; Proposition 2 is the proof.

The adversary splits a fixed budget across k identities (E4). The escape ratio of audit-only and token-bucket policies rises linearly with k, from 1.07 at k=12 to 2.43 at k=60: audit-only has no key, and the bucket refills 180ε over the one-hour trace against a 0.3ε fragment, so it never binds. Static quota saturates at 1.30 and exact blocking reaches 1.69; neither grows with k once the split exceeds what the per-platform counters absorb. PrivFuse is flat at 0.82 across the range and exact global is flat at 0.98, because neither can be made to forget that the fragments concern the same subject. The flatness, not the level, is the result: the attack's only lever is the identity it uses, and identity is not part of the key.

Exact global is the strongest baseline and it is sound: at k=42 it admits 0.929 against 0.923 and leaves 2% of the cap unused against 18%. The 18% is the residue of the soft threshold—PrivFuse throttles from $0.8E$ and throttled queries consume half a charge—so the worst subject ends below its cap rather than at it. What exact global lacks is a window,

a degradation path and an estimator whose memory does not grow with N: the split result shows that subject keying is necessary and that PrivFuse achieves it, not that PrivFuse dominates exact global on every axis.

E. Where Is the SLO Frontier?

Sweeping the soft threshold α from 0.55 to 0.95 at four values of p moves availability from 0.749 to 0.794 and raises the fraction of settings in which the *monitored* violation criterion is met from 0.25 to 1.0, while the hard cap never moves. The knobs are not independent: at $\alpha{=}0.55$ the monitored rate is 2.2%, so the criterion holds only for $p \geq 5\%$, while at $\alpha \geq 0.85$ it is at most 0.22% and the default $p{=}1\%$ holds—a tight target and a loose rate are 4.5 points apart, and E25 shows where the population rate departs from the monitored one.

The sensitivity sweep varies one factor at a time with one seed per point, so it locates trends, not variability. Two points bracket aggregation: with one platform, static quota becomes sound—worst spend 0.969ε—because a per-platform quota on the only platform is also a per-subject quota; with six, its worst spend rises to 1.859ε while PrivFuse stays at 0.825ε. A heavier tail ($\alpha_{\text{zipf}}{=}1.5$) costs 15 points of availability, from 0.936 to 0.783; 200 tenants cost 4; a 64-entry hot table costs 2; moving δ from 10^{-6} to 10^{-4} raises availability from 0.920 to 0.941; and crashes at 0.05 cost one point. Across all 19 points violations are zero and the estimator never under-estimates.

F. Ablation and Stability

Removing one component at a time (three seeds, E7): the full system admits 0.923 of requests at 4.8 MB and 100.9 μs, exact global 0.929 at 0.24 MB and 10.8 μs. Removing degradation, so the breaker can only admit or refuse, costs 4.2 points of availability: degradation converts would-be refusals into answers. Shrinking the sketch from 2^{14} to 2^9 columns cuts memory to 0.71 MB without moving availability, and widening it only costs memory—on 1,500 subjects the hot table holds everyone who matters.

Churn is the cost the capacity sweep hides (E14): rotating 400 heavy subjects per epoch raises the mean over-estimate from 0.16 to 0.28 at 0.15ε and from 0.25 to 0.65 at 1.0ε, and the p99 from 1.7 to 5.1×. Safety is untouched, since an evicted subject is re-seeded from the sketch, but the table stops paying for itself.

Disabling the table raises availability to 0.930 but widens every windowed estimate. That baseline is less comfortable than it suggests: at 1,500 subjects exact cumulative accounting matches PrivFuse's availability within a point (0.898 against 0.906 at 1.0ε) with 0.38 MB against 5.05 MB in the same 600 s configuration, and its 19 μs runs no breaker, coordinator or window. A windowed exact accountant (E16) admits every request at 0.5ε and still lets six of 1,500 subjects exceed their *lifetime* cap, spending 1.08 MB; at 1.0ε it reaches 1.000 against 0.906 with one lifetime violation. Putting the exact table behind the same ring, breaker and coordinator isolates approximation from functionality (E20): availability and windowed peak are equal at both caps, and the exact version is faster, 156 against 233 μs, at 0.37 against 5.01 MB. At 10^3 subjects the sketch buys nothing but the slope; measured from 10^3 to 10^6 with four live tables, it ends at 1.2 GB resident against 10.6 MB (E23).

E26 adds a sparse exact map—one ring per active subject—and the settlement state behind all three, at 10^6 subjects with four epochs: 30.5 MB dense, 1.6, 17.5, 83.8 and 168 MB sparse at 1, 10, 50 and 100% activity, and 2.7 MB as a sketch. Below about 2% activity the sparse map is both smaller and exact, and we do not claim the sketch there; by 10% the sketch is six times smaller. In the full system the exact ledger costs about 1.1 KB per committed subject, so at full activity the settlement state alone is 1.1 GB and the estimator choice moves the total by a fraction of a percent: the sketch earns its place in the window state, not the settlement state.

On a load oscillating around the threshold, hysteresis and dwell give the breaker zero transitions; without them, 26, of which 13 are reversals.

G. Why zCDP and Not Rényi DP

The choice of algebra is about solvability, not tightness. A Rényi accountant evaluates a discrete grid of orders and its conversion to (ε, δ) floors at $\log(1/\delta)/(\alpha_{\max} - 1)$, which is 0.0225 at $\delta = 10^{-5}$ and $\alpha_{\max}{=}512$: a tenant asking for $\varepsilon_q{=}0.02$ sits below that floor, and our implementation saturates at $\sigma{=}1000$, 4.17 times the zCDP scale, and refuses the query rather than charge 0.023. Basic composition reaches 0.2008 after 100 charges at that target, ten times zCDP's budget; at $\varepsilon_q{=}0.1$ and 0.5 the three backends agree to 0.004% and 0.006%. A Rényi grid extended to $\alpha_{\max} \geq 1{,}200$ would also calibrate 0.02, at the price of a conversion admission must search rather than invert.

H. What Soundness Costs in Utility

The utility experiment puts a real dataset behind the cap: the UCI breast-cancer set (569 records, 30 features), split 70/30 stratified into 398 training and 171 test records. Features are clipped to the public unit interval, so the L2 sensitivity of a mean vector is $\sqrt{30}/398 = 3.4 \times 10^{-3}$; each release is four Gaussian mechanisms at a quarter of the per-release zCDP cost, the variance is debiased by the released noise variance, and five seeds give the spread. At $\varepsilon_q{=}1.0$ PrivFuse admits one query at 0.87 ± 0.06; at 0.5ε three at 0.84 ± 0.06; at 0.25ε fifteen at 0.60 ± 0.22. Cheaper releases do not buy accuracy: the admissible count grows as $1/\varepsilon_q^2$, so the $\sqrt{n}$ averaging gain cancels the $1/\varepsilon_q$ noise. The baselines answer 60 or 100 queries at 0.92–0.93 while spending 2.0–12ε, buying accuracy with a violation in every seed; 13.7% of admitted queries are throttled rather than refused.

I. Settlement and Concurrency

Six settlement invariants pass, among them the refusal to re-bind a reserve: the identifier is bound to a fingerprint of tenant, platform, query, support, charge, mechanism and policy

version, and a per-identifier lock serialises its owners before the stripes are taken, so two requests with disjoint supports cannot both insert a reservation. A committed transaction replays its stored receipt, and a second commit carrying a different receipt is refused. In the atomicity test, 16 threads issue 3,200 attempts against a subject whose cap funds 100 charges; every thread count admits exactly 100, 1.0ε and zero violations.

Injecting a crash at each step of the transaction tests Proposition 3 directly (E19), with the invariants read off the ledger. Over every crash point, one and four threads and a crash rate up to 5%, no answer left without a committed charge, no transaction was charged twice, and in-flight spend never crossed the cap: a crash after reserve with a same-id retry delivers once and charges once, a retry after the TTL finds the hold returned, and a crash after commit withholds the answer but keeps the charge. The price is the crash rate, 188 of 200 delivered at 5%, plus one request in 800 blocked by a suspended hold; three unshared per-platform ledgers let a subject reach $3.0\times$ its cap.

E19 keeps the ledger alive; E21 kills the process. Six SIGKILLs at the four settlement steps deliver 389 answers, none uncharged, at 0.191ε against a 0.2ε cap; the same schedule on an in-memory ledger delivers 571 and pushes five of ten subjects past the cap—a restart that forgets is a budget reset. Durability costs $76\,\mu$s per transition against $7.5\,\mu$s. Striping pays only for disjoint subjects (E13, medians of three): four threads reach 21.4k operations per second on one subject against 26.5k for a global lock, and 91.5k on disjoint subjects—it removes a serialisation point, not contention.

Three platform processes against a coordinator of their own reach about 230 answers per second with each step fsynced (E22); killing that coordinator mid-trace and restarting it from the log costs $\approx$1.1 s and a few percent of throughput, and still delivers 760 answers, none uncharged, no duplicate charge, worst client spend 0.38ε against 0.5ε. One client over loopback sustains 1.9k pairs per second at a $201\,\mu$s median round trip. The caveats are one host, a loopback trip and a stub adapter that hides the real commit trip.

VIII. Related Work

Privacy accounting. Recent work on continual observation separates oblivious from adaptive adversaries [16] and revisits the cost of online queries [17]. Tight auditing has become its own line, from reconstruction attacks on production query systems [7] to Gaussian-DP audits of synthetic data [8] and analyses of the gap between provable and auditable privacy in DP-SGD [9]. All of it asks how large the loss was; PrivFuse asks whether the next release may happen. We do not improve a composition bound: we take zCDP [5] and make it cheap and monotone enough for admission, measuring Rényi [4] and Gaussian DP [11] against it.

Systems that enforce privacy. PINQ established the pattern of a query interface that charges a budget per analyst [18]; Fuzzi and Flex turn it into a deployed SQL system with elastic sensitivity and per-query pricing [19], and production libraries apply the same accountant to training [20], [21]. Cohere tracks DP budgets for analytics pipelines at block granularity [22]; PrivateKube schedules one budget across tenants [23] and DProvDB tracks provenance across colluding analysts [24]. All of them key the budget to an analyst, an application or a block inside one deployment, so none charges a *subject* whose records sit on several platforms or treats the resulting counter as a transaction. Prochlo moves enforcement into an enclave and removes the trusted server, fixing the unit to the user and the granularity to a batch [25], a model now studied formally as anonymous shuffling [26]; IncShrink combines MPC with DP at a cryptographic protocol per query [27].

Auditing and governance. A parallel literature builds post-market monitoring [28] and ethics-based audit [29], and empirical studies show how far declared policy is from measured behaviour [6], now extending to AI-mediated platforms [30]. PrivFuse assumes the declaration exists and makes it enforceable at the point of sale.

Data markets. Trustless trading has occupied the blockchain systems community, with recent designs scaling delivery and settlement [1] and provenance mechanisms tracking derived artifacts [12]. These answer "did I receive what I paid for?". Pricing privacy has been studied as a negotiation between subject and buyer [31], presupposing a budget that can be spent and tracked; the prior question—"may this answer be sold at all?"—is orthogonal. We add a budget root that every platform consults and make the charge itself a two-phase transaction keyed by the subject across platforms.

Approximate counting. Count-Min sketches and their conservative variants are well understood [13], [14], including on power-law streams, and learned variants reduce error at the same width [15]; we want an error with a known sign.

Mechanisms. Every mechanism we price exposes a cost vector: DP synthesis [32], spectral perturbation [33], geometric preconditioning [34], [35]. How the vector is produced changes the vector, not the enforcement layer.

IX. Discussion

Three principles transfer. Split the accounting state by the question it answers: exact where the quantity is money and must be settled, one-sided where it has time structure and must be cheap. Key every charge on the party that bears the loss, and treat the charge as a transaction with a commit point.

A deployment would replicate the coordinator over consensus nodes, own each subject on one node so the critical section stays local, and shard by subject range; E22 takes the first step with separate processes on one host and a loopback network, so we claim a trusted-coordinator prototype only. A subject's budget would then live on one node, which could under-report a charge; receipts make that detectable only afterwards, and closing it needs Byzantine agreement or a proof that the charge matches the mechanism executed. The accounting layer is itself sensitive—it records which subjects appear in which queries—and accumulators, oblivious access or an enclave each cost something the design does not budget for.

Two further limitations bound the claims. The monitor is exact over charged subjects, so its coverage depends only on whether a violation was charged, and E25 measures that loss (recall 0.014 with an eight-slot table on flat popularity). Of the two adversaries measured, one re-times its fragments after observing throttling (E18, loss 0.80 to 0.85 of the cap) and one cycles subjects through the hot table (E14, costing accuracy but never soundness).

Two conditions would falsify the approach. For a small population under a cumulative policy, exact counting is cheaper (our ablation, $1{,}500$ subjects), and with one platform a subject-keyed quota is sound while a tenant-keyed one is not. If a query's cost cannot be predicted, admission must price conservatively, weakening the guarantee to what auditing provides; the design is for what remains, many subjects over several platforms, where the promise must hold before the answer leaves the building.

X. Conclusion

Privacy budgets in data markets are enforced after the fact, keyed on the party that pays and reconciled outside the transaction that spends them, so a tenant can split its traffic, release before the overspend is known, and lose the charge on a crash. PrivFuse moves the decision to admission around three commitments—an exact ledger, a one-sided sketch, and a two-phase reservation that separates a budget from a guess. On a million subjects the estimator holds 10.6 MB where exact accounting needs 1.2 GB; enforcement admits 0.93 of requests at 1.0ε with no violations where per-platform blocking admits 0.98 and breaches the cap a dozen times; and the escape ratio under a splitting tenant stays at 0.82 where policies without a subject key reach 2.43. The cost, 4.4 points of availability, is reported with the gains. Next comes privatising the accounting state and replacing trust in one coordinator with a proof that the charge matches the mechanism that ran.

References

[1] X. Liu, Z. Guo, L. Liu, M. Zheng, Y. Qiu, and L. Jiang, "Yotta: A large-scale trustless data trading scheme for blockchain system," Nanyang Blockchain Conference 2025, 2025.

[2] F. Fioretto, P. V. Hentenryck, and J. Ziani, "Differential privacy overview and fundamental techniques," arXiv:2411.04710, 2024.

[3] D. Desfontaines and B. Pejó, "SoK: Differential privacies," *Proceedings on Privacy Enhancing Technologies*, no. 2, 2020.

[4] I. Mironov, "Rényi differential privacy," in *2017 IEEE 30th Computer Security Foundations Symposium (CSF)*. IEEE, 2017, pp. 263–275.

[5] M. Bun and T. Steinke, "Concentrated differential privacy: Simplifications, extensions, and lower bounds," arXiv:1605.02065, 2016.

[6] A. Kapp, "Collection, usage and privacy of mobility data in the enterprise and public administrations," *Proceedings on Privacy Enhancing Technologies*, vol. 2022, no. 4, pp. 440–456, 2022.

[7] A. Cohen and K. Nissim, "Linear program reconstruction in practice," arXiv:1810.05692, 2018.

[8] G. Ganev, M. S. M. S. Annamalai, and B. Kulynych, "Tight auditing of differential privacy in MST and AIM," arXiv:2604.18352, 2026.

[9] S. Modi, S. Balaji, Y. Zhu, G. Kamath, and K. Tian, "Revisiting the provable-auditable privacy gap of DP-SGD," arXiv:2608.28934, 2026.

[10] C. Dwork, "Differential privacy," in *Automata, Languages and Programming (ICALP)*, ser. Lecture Notes in Computer Science, vol. 4052. Springer, 2006, pp. 1–12.

[11] J. Dong, A. Roth, and W. J. Su, "Rejoinder: Gaussian differential privacy," arXiv:2104.01987, 2021.

[12] J. Zhu and L. Wang, "Auditing data provenance in real-world text-to-image diffusion models for privacy and copyright protection," arXiv:2506.11434, 2025.

[13] É. Fusy and G. Kucherov, "Count-min sketch with variable number of hash functions: an experimental study," arXiv:2302.05245, 2023.

[14] ——, "Phase transition in count approximation by Count-Min sketch with conservative updates," in *Approximation and Online Algorithms (WAOA 2022)*, ser. Lecture Notes in Computer Science. Springer, 2023, pp. 232–246.

[15] K. Nishishita, A. Sato, and Y. Matsui, "Optimized learned count-min sketch," arXiv:2512.12252, 2025.

[16] M. Bun, M. Gaboardi, and C. Wagaman, "Separating oblivious and adaptive differential privacy under continual observation," arXiv:2603.11029, 2026.

[17] M. Bun, T. Steinke, and J. Ullman, "Make up your mind: The price of online queries in differential privacy," arXiv:1604.04618, 2016.

[18] H. Ebadi and D. Sands, "Featherweight PINQ," arXiv:1505.02642, 2015.

[19] N. Johnson, J. P. Near, and D. Song, "Towards practical differential privacy for SQL queries," *Proceedings of the VLDB Endowment*, vol. 11, no. 5, pp. 526–539, 2018.

[20] A. Yousefpour, I. Shilov, A. Sablayrolles, D. Testuggine, K. Prasad, M. Malek, J. Nguyen, S. Ghosh, A. Bharadwaj, J. Zhao, G. Cormode, and I. Mironov, "Opacus: User-friendly differential privacy library in PyTorch," arXiv:2109.12298, 2021.

[21] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, and L. Zhang, "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*. ACM, 2016, pp. 308–318.

[22] N. Küchler, E. Opel, H. Lycklama, A. Viand, and A. Hithnawi, "Cohere: Managing differential privacy in large scale systems," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 991–1008.

[23] T. Luo, M. Pan, P. Tholoniat, A. Cidon, R. Geambasu, and M. Lécuyer, "Privacy budget scheduling," in *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*. USENIX Association, 2021.

[24] S. Zhang and X. He, "DProvDB: Differentially private query processing with multi-analyst provenance," in *Proceedings of the ACM on Management of Data*, vol. 1, no. 4. ACM, 2023, pp. 1–27.

[25] A. Bittau, Ú. Erlingsson, P. Maniatis, I. Mironov, A. Raghunathan, D. Lie, M. Rudominer, U. Kode, J. Tinnes, and B. Seefeld, "Prochlo: Strong privacy for analytics in the crowd," in *Proc. 26th Symp. Operating Systems Principles (SOSP)*. ACM, 2017, pp. 441–459.

[26] A. Cheu, A. Smith, J. Ullman, D. Zeber, and M. Zhilyaev, "Distributed differential privacy via shuffling," in *Advances in Cryptology – EUROCRYPT 2019*, ser. Lecture Notes in Computer Science, vol. 11476. Springer, 2019, pp. 375–403.

[27] C. Wang, J. Bater, K. Nayak, and A. Machanavajjhala, "Incshrink: Architecting efficient outsourced databases using incremental MPC and differential privacy," in *Proc. ACM SIGMOD*. ACM, 2022, pp. 818–832.

[28] J. Mökander, M. Axente, F. Casolari, and L. Floridi, "Conformity assessments and post-market monitoring: A guide to the role of auditing in the proposed European AI regulation," *Minds and Machines*, vol. 32, no. 2, pp. 241–268, 2021.

[29] J. Mökander, J. Morley, M. Taddeo, and L. Floridi, "Ethics-based auditing of automated decision-making systems: Nature, scope, and limitations," *Science and Engineering Ethics*, vol. 27, no. 4, 2021.

[30] A. Shouli, A. Barthwal, M. Campbell, and A. K. Shrestha, "Ethical AI for young digital citizens: A call to action on privacy governance," *Security and Privacy*, vol. 9, no. 2, p. e70202, 2026.

[31] K. Jung and S. Park, "Privacy bargaining with fairness: Privacy-price negotiation system for applying differential privacy in data market environments," in *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 2019, pp. 1389–1394.

[32] U. Saha, T. M. Tonoy, and H. Imtiaz, "DP-CDA: An algorithm for enhanced privacy preservation in dataset synthesis through randomized mixing," *Security and Privacy*, vol. 9, no. 2, p. e70207, 2026.

[33] C. Feng, N. Xu, W. Wen, P. Venkitasubramaniam, and C. Ding, "Spectral-DP: Differentially private deep learning through spectral perturbation and filtering," arXiv:2307.13231, 2023.

[34] M. M. V. den Bosch, R. Taiello, A. S. Aillet, A. Protani, M. A. G. Ballester, and L. Serio, "DP-KFC: Data-free preconditioning for privacy-preserving deep learning," arXiv:2605.13418, 2026.

[35] J. Duan, H. Hu, Q. Ye, and X. Sun, "Technical report: Full version of analyzing and optimizing perturbation of DP-SGD geometrically," Proc. IEEE ICDE, 2025.